

Practice 5.

Guillem Moreno Garcia

Objectives:

1. Understand the projection methods for classification.
2. Understand the Fisher discriminant.
3. Do a simple exploratory data analysis
4. Learn the **data cleaning and visualization** features of Pandas.

Application:

Data base: Data set from the UCI Machine Learning Repository (<http://archive.ics.uci.edu/ml>)
(<http://archive.ics.uci.edu/ml>)

Database: Wine quality, <http://archive.ics.uci.edu/ml/datasets/Wine+Quality>
(<http://archive.ics.uci.edu/ml/datasets/Wine+Quality>).

- Predict **quality** (score between 0 and 10)

```
In [1]: import pandas as pd
import os
import matplotlib.pyplot as plt
import numpy as np
import sklearn
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
```

```
In [2]: File = "winequality-red.csv"
Filename = os.path.join(os.getcwd(), 'Data', File)
print(f'Filename with path: \n {Filename}')
Data = pd.read_csv(Filename)
Data.head().T
```

Filename with path:
c:\mlearn-lab\FifthPractice\Data\winequality-red.csv

Out[2]:

	0	1	2	3	4
fixed acidity	7.4000	7.8000	7.800	11.200	7.4000
volatile acidity	0.7000	0.8800	0.760	0.280	0.7000
citric acid	0.0000	0.0000	0.040	0.560	0.0000
residual sugar	1.9000	2.6000	2.300	1.900	1.9000
chlorides	0.0760	0.0980	0.092	0.075	0.0760
free sulfur dioxide	11.0000	25.0000	15.000	17.000	11.0000
total sulfur dioxide	34.0000	67.0000	54.000	60.000	34.0000
density	0.9978	0.9968	0.997	0.998	0.9978
pH	3.5100	3.2000	3.260	3.160	3.5100
sulphates	0.5600	0.6800	0.650	0.580	0.5600
alcohol	9.4000	9.8000	9.800	9.800	9.4000
quality	5.0000	5.0000	5.000	6.000	5.0000

```
In [3]: Data.dtypes
print(f'Shape of the database:{Data.shape}')
```

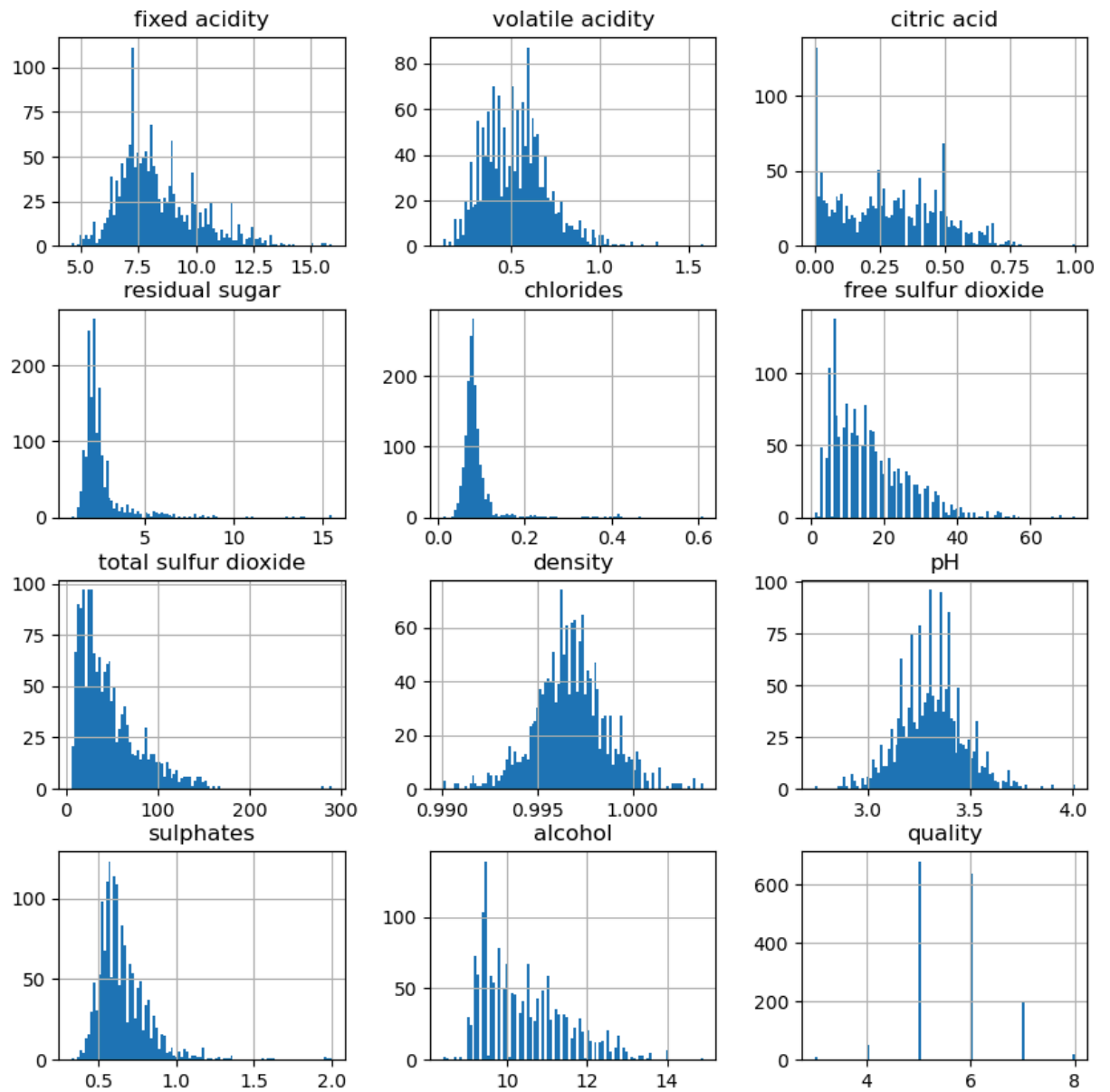
Shape of the database:(1599, 12)

```
In [4]: Data.describe().round(2).T
```

Out[4]:

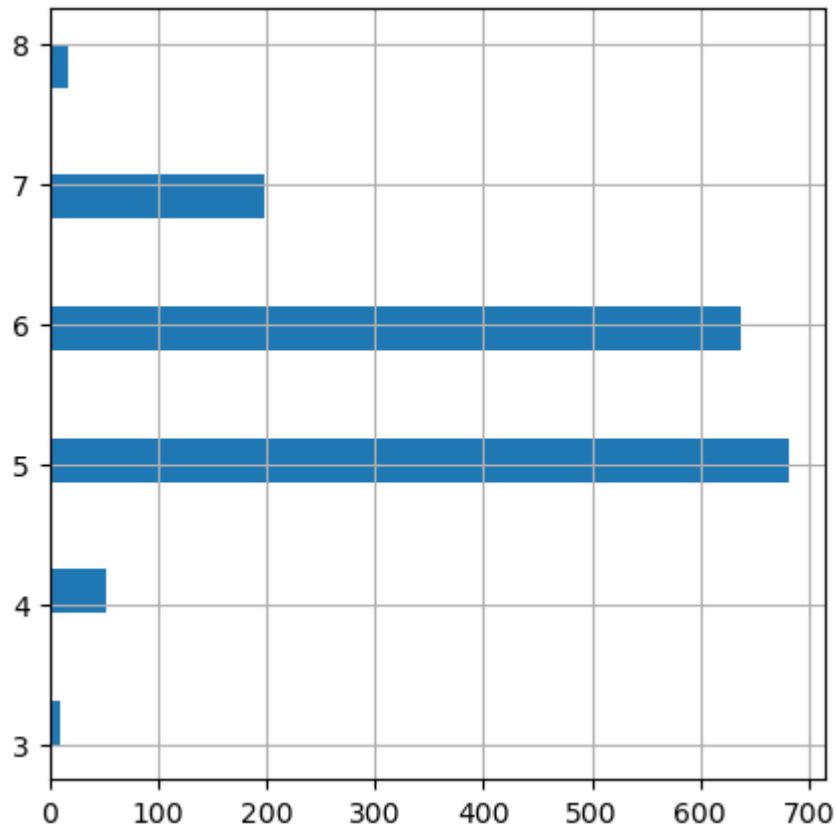
	count	mean	std	min	25%	50%	75%	max
fixed acidity	1599.0	8.32	1.74	4.60	7.10	7.90	9.20	15.90
volatile acidity	1599.0	0.53	0.18	0.12	0.39	0.52	0.64	1.58
citric acid	1599.0	0.27	0.19	0.00	0.09	0.26	0.42	1.00
residual sugar	1599.0	2.54	1.41	0.90	1.90	2.20	2.60	15.50
chlorides	1599.0	0.09	0.05	0.01	0.07	0.08	0.09	0.61
free sulfur dioxide	1599.0	15.87	10.46	1.00	7.00	14.00	21.00	72.00
total sulfur dioxide	1599.0	46.47	32.90	6.00	22.00	38.00	62.00	289.00
density	1599.0	1.00	0.00	0.99	1.00	1.00	1.00	1.00
pH	1599.0	3.31	0.15	2.74	3.21	3.31	3.40	4.01
sulphates	1599.0	0.66	0.17	0.33	0.55	0.62	0.73	2.00
alcohol	1599.0	10.42	1.07	8.40	9.50	10.20	11.10	14.90
quality	1599.0	5.64	0.81	3.00	5.00	6.00	6.00	8.00

```
In [5]: ax = Data.hist(figsize= (10,10),bins = 100)
```



```
In [6]: ax = Data['quality'].hist(figsize= (5,5),orientation="horizontal",bins = 16)
print('Note: predicted qualities are **unbalanced** ')
```

Note: predicted qualities are **unbalanced**

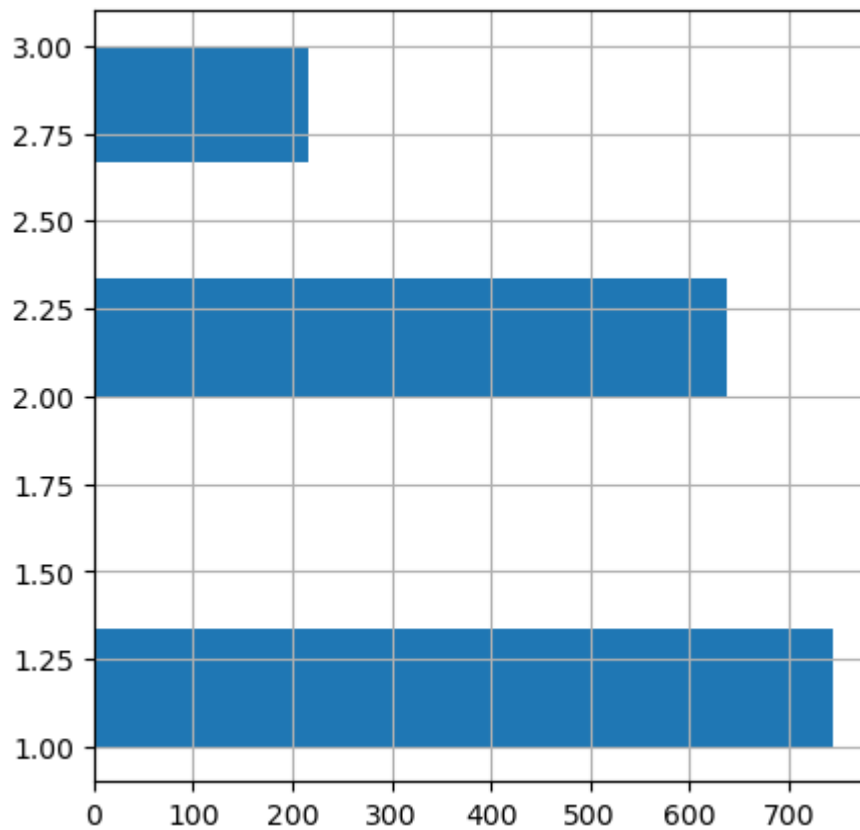


Lets reduce the number of classes to three: low, medium, high

```
In [7]: CodingQuality = {3:1,4:1,5:1,6:2,7:3,8:3}
Data['quality'] = Data['quality'].map(CodingQuality)
```

```
In [8]: ax =Data['quality'].hist(figsize= (5,5),orientation="horizontal",bins = 6)
print('Note: predicted qualities are almost balanced ')
```

Note: predicted qualities are almost balanced

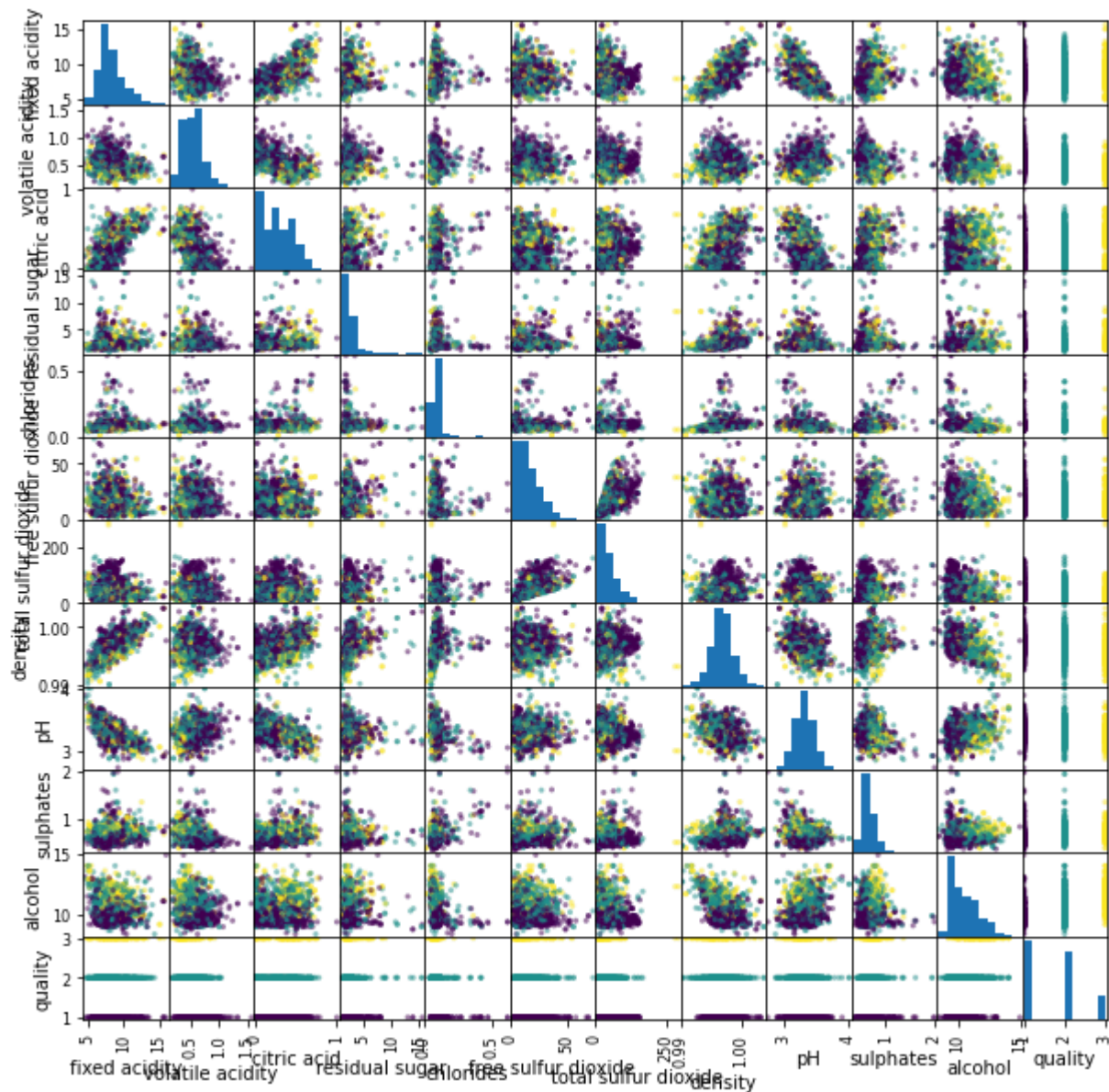


```
In [ ]:
```

Scatter plot.

Note that the class quality is coded with a color, i.e. `c =Data['quality']`

```
In [9]: if False:
        ax = pd.plotting.scatter_matrix(Data,c =Data['quality'], figsize = (10,10))
```



Partition database into train and test

Note the helper function

```
In [10]: def PartitionOfDatabase(Data, Fraction = 0.75):
    NumSamples, NumFeatures = Data.shape
    Index = list(range(NumSamples))
    RandomIndex = np.random.permutation(Index)
    NumTrainElements = int(Fraction*NumSamples)
    TrainIndex = RandomIndex[:NumTrainElements]
    TestIndex = RandomIndex[NumTrainElements:]
    TrainDB = Data.iloc[TrainIndex]
    TestDB = Data.iloc[TestIndex]
    return TrainDB, TestDB
TrainDB, TestDB = PartitionOfDatabase(Data, Fraction = 0.75)
```

```
In [11]: y = TrainDB['quality']
InputFeatures = [fea for fea in TrainDB.columns if fea != 'quality']
print(f' Input variables: \n {InputFeatures}')
print('_'*50)
X = TrainDB[InputFeatures]
```

Input variables:
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol']

Next we standarize the data, so that it has mean zero and standard deviation equal to one.

$$z = \frac{(x - \text{mean})}{\text{std}}$$

Important: Use the same scaler for the test database

```
In [12]: scaler = StandardScaler()
print(scaler.fit(X))
X = scaler.transform(X)
X = pd.DataFrame(data = X, columns = InputFeatures)
X.describe().round(2).T
print('** Important: Use the same scaler for the test database!**')
```

StandardScaler()
** Important: Use the same scaler for the test database!**

```
In [13]: X_test = TestDB[InputFeatures]
y_test = TestDB['quality']
X_test = scaler.transform(X_test)
X_test = pd.DataFrame(data = X_test, columns = InputFeatures)
print('Test database standardized with the training scaler.')
```

Test database standardized with the training scaler.

We create the projection into 2D by the two methods.

```
In [14]: Dimension_of_projection = 2
```

```
In [15]: lda = LinearDiscriminantAnalysis(n_components=Dimension_of_projection)
X_lda = lda.fit(X, y).transform(X)

X_lda = pd.DataFrame(data = X_lda, columns = ['dim1', 'dim2'])
```

```
In [16]: X_test_lda = lda.transform(X_test)
X_test_lda = pd.DataFrame(data = X_test_lda, columns = ['dim1', 'dim2'])
print('Test database projected with the same LDA model.')
```

Test database projected with the same LDA model.

Homework: State briefly the criterion for deriving each method and the underlying hypothesis.

PCA chooses directions of maximum variance without using the labels, so its criterion is purely geometric. LDA/Fisher chooses directions that maximize the ratio of between-class scatter to within-class scatter, so it uses the class labels and assumes the classes are approximately Gaussian with a common covariance matrix.

Homework: Given a multidimensional Gaussian with $x \in \mathbb{R}^p$, which is the distribution of a linear combination of the dimensions of x ?

2. For a multivariate Gaussian, any linear combination of the coordinates is still Gaussian.

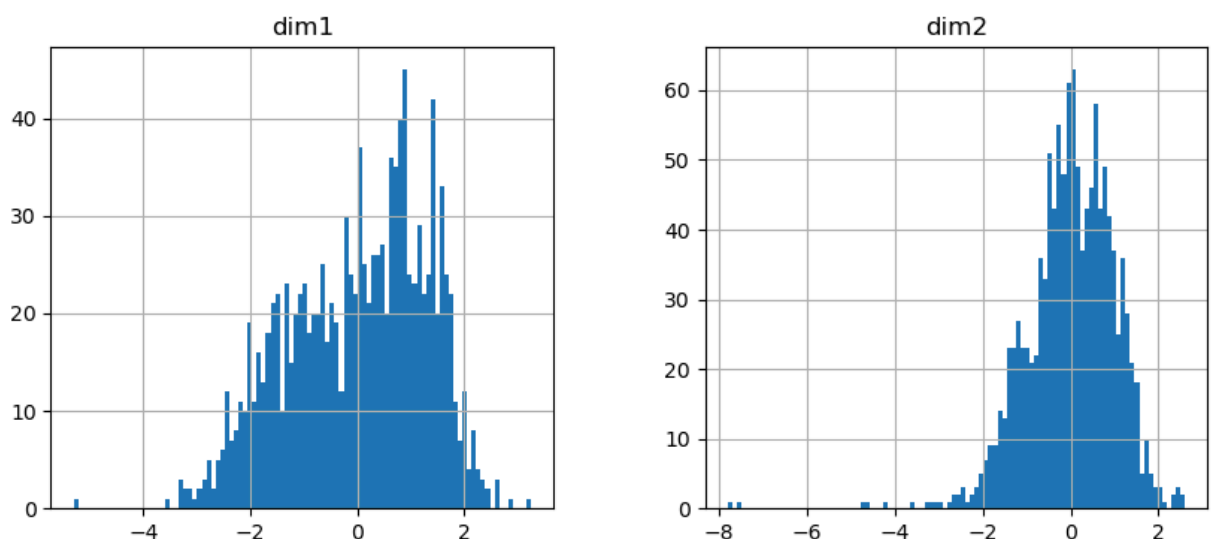
Description of the projected elements

```
In [17]: X_lda.describe().round(2).T
```

Out[17]:

	count	mean	std	min	25%	50%	75%	max
dim1	1199.0	-0.0	1.29	-5.3	-0.99	0.15	1.01	3.24
dim2	1199.0	-0.0	1.01	-7.8	-0.54	0.06	0.70	2.60

```
In [18]: ax = X_lda.hist(figsize = (10,4),bins = 100)
```



Homework: Give a summary by words of the statistical distributions.

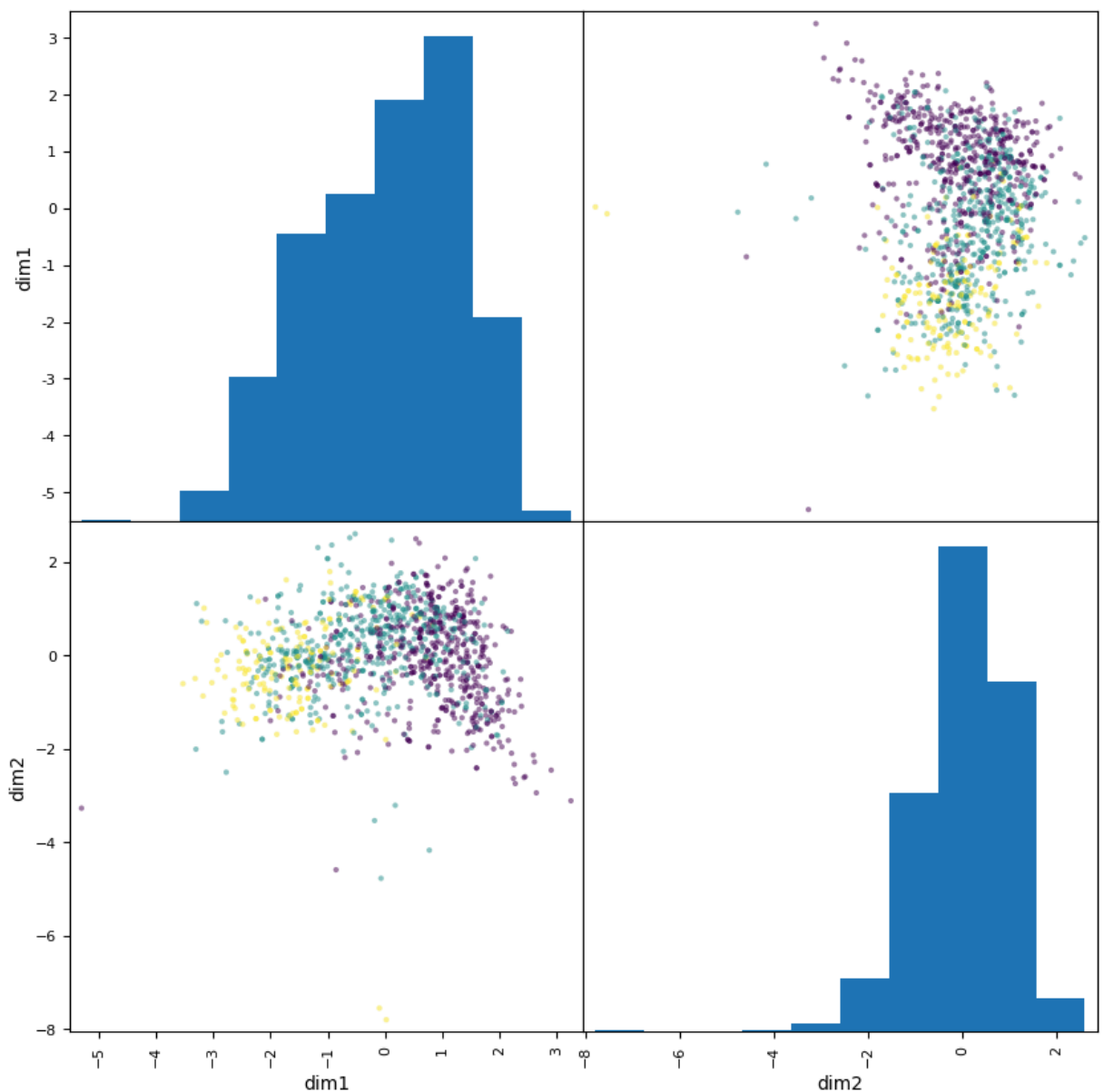
1. Is the mean different from the percentile 50%.
2. Is the std deviation comparable to the interquartile distance (i.e. difference between the percentile 75% and the percentile 25%)
3. How many stds are the minimum and maximum from the mean value. Is it likely for a Gaussian?
4. Do the hypothesis of the methods stand?.

1. The mean is close to the median in `dim1`, but `dim2` is more skewed, so the two are not identical there.
2. The standard deviation and the interquartile range are of the same order, although `dim2` has a longer positive tail.
3. The extremes are a few standard deviations away from the mean; `dim2` has a clear positive outlier around 7.6 standard deviations.
4. The projection is a reasonable first approximation, but the distributions are not perfectly Gaussian.

Lets look into the space of the projection

$$P(.) : x \in \mathbb{R}^{11} \longrightarrow \mathbb{R}^2$$

```
In [19]: ax = pd.plotting.scatter_matrix(X_lda,c =y, figsize = (10,10))
```



```
In [20]: lda.coef_[ :1, :].shape
```

```
Out[20]: (1, 11)
```

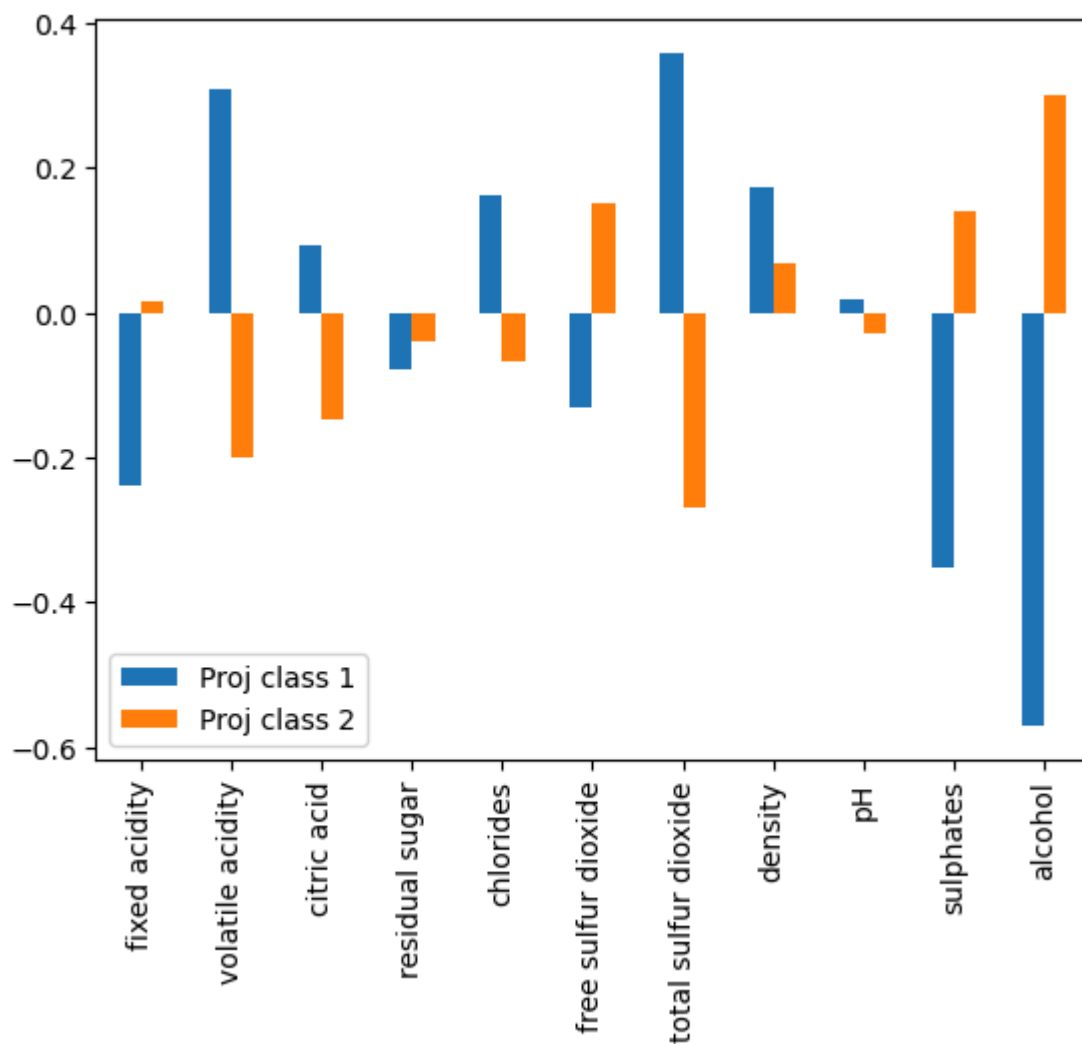
```
In [21]: Coef_Lda = pd.DataFrame(data = lda.coef_[:2,:].T, columns = ['Proj class 1', 'Proj class 2'], index = X.columns.values)
Coef_Lda
```

Out[21]:

	Proj class 1	Proj class 2
fixed acidity	-0.238835	0.016374
volatile acidity	0.308210	-0.199461
citric acid	0.093590	-0.149150
residual sugar	-0.077880	-0.040063
chlorides	0.160816	-0.066310
free sulfur dioxide	-0.132458	0.151710
total sulfur dioxide	0.357739	-0.269803
density	0.173124	0.068341
pH	0.018074	-0.029749
sulphates	-0.352432	0.141002
alcohol	-0.571056	0.298794

```
In [22]: ax = Coef_Lda.plot.bar()
print('Check what would have happened if we had not standardized.')
```

Check what would have happened if we had not standardized.



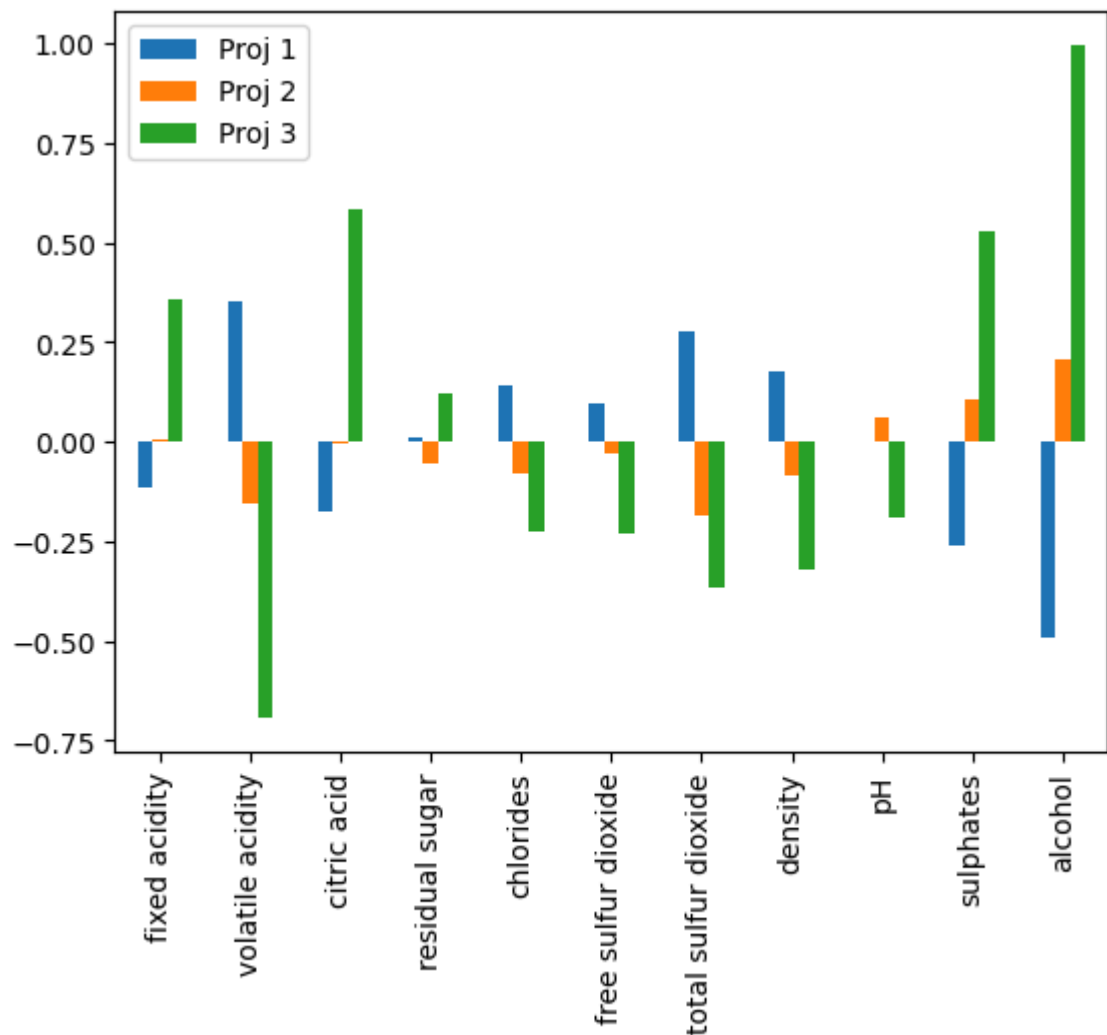
```
In [23]: print('Lets see how the means of each class compare to each other')
Means_Lda = pd.DataFrame(data = lda.means_.T, columns = ['Proj 1', 'Proj 2', 'Proj 3'], index = X.columns.values)
Means_Lda
```

Lets see how the means of each class compare to each other

Out[23]:

	Proj 1	Proj 2	Proj 3
fixed acidity	-0.113896	0.004000	0.355123
volatile acidity	0.352021	-0.152971	-0.693228
citric acid	-0.175476	-0.006421	0.583311
residual sugar	0.012986	-0.056581	0.120909
chlorides	0.140099	-0.077805	-0.227224
free sulfur dioxide	0.097169	-0.027844	-0.232710
total sulfur dioxide	0.279378	-0.185239	-0.366599
density	0.176917	-0.085965	-0.322273
pH	0.002685	0.062546	-0.188509
sulphates	-0.259750	0.106350	0.530285
alcohol	-0.493401	0.206234	0.995152

```
In [24]: ax = Means_Lda.plot.bar()
```



Homework:

1. Comment on the meaning of the coefficients. What is the result of having different signs and sizes of the values?
 2. If you have time, what would have happened if we had not standardized? What is the influence of the size/order of magnitude of the features?
 3. What can you say about the means in the sense of separation between classes? Note that there seems to be like a continuity between the centers of mass for each quality.
-
1. The sign tells the direction of the effect and the magnitude tells how strongly a feature influences the discriminant score.
 2. Because the features were standardized, the coefficients are comparable across variables; without standardization, variables with larger numeric scale would dominate only because of their units.
 3. The strongest features are the ones tied to wine style and balance, especially alcohol, volatile acidity, sulphates, and sulfur dioxide. The class means move gradually from low quality to high quality rather than forming three completely separate clusters, so there is a clear continuity between the centers of mass for the three classes.

Homework:

1. Create three logistic linear classifiers: Class 1 vs. {Class 2 and class 3}, Class 2 vs. {Class 1 and class 3}, Class 3 vs. {Class 1 and class 2} **Is there a difference between the performance on train and test?**
2. Do the classification on the original data, and on the data after the **lda** projection
3. From the coefficients of the lda and logistic linear classifiers, and the understanding of the problem, can you think of a feature engineering that might improve the problem
4. **Remember: the 'l2' option in the logistic classifier does a regularization function.**

```

In [26]: from IPython.display import display
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier

def one_vs_rest_summary(X_train, X_test, y_train, y_test):
    rows = []
    for cls in sorted(y_train.unique()):
        y_train_binary = (y_train == cls).astype(int)
        y_test_binary = (y_test == cls).astype(int)
        model = LogisticRegression(max_iter=2000, solver='liblinear', random_state=0)
        model.fit(X_train, y_train_binary)
        rows.append({
            'class': cls,
            'train_accuracy': accuracy_score(y_train_binary, model.predict(X_train)),
            'test_accuracy': accuracy_score(y_test_binary, model.predict(X_test))
        })
    return pd.DataFrame(rows)

original_results = one_vs_rest_summary(X, X_test, y, y_test)
lda_results = one_vs_rest_summary(X_lda, X_test_lda, y, y_test)

summary = pd.concat(
    [
        original_results.assign(space='original'),
        lda_results.assign(space='lda')
    ],
    ignore_index=True
)

print('One-vs-rest logistic regression accuracy by class and feature space:')
display(summary.pivot(index='class', columns='space', values=['train_accuracy', 'test_accuracy']).round(4))

overall_original = OneVsRestClassifier(LogisticRegression(max_iter=2000, solver='liblinear', random_state=0))
overall_original.fit(X, y)
overall_lda = OneVsRestClassifier(LogisticRegression(max_iter=2000, solver='liblinear', random_state=0))
overall_lda.fit(X_lda, y)

print(f'Overall OVR accuracy on original data: train={accuracy_score(y, overall_original.predict(X)):.4f}, test={accuracy_score(y_test, overall_original.predict(X_test)):.4f}')
print(f'Overall OVR accuracy on LDA projection: train={accuracy_score(y, overall_lda.predict(X_lda)):.4f}, test={accuracy_score(y_test, overall_lda.predict(X_test_lda)):.4f}')
print('Interpretation: train scores are slightly higher than test scores, and the original feature space is only marginally better than the 2D LDA projection.')
print('Feature-engineering idea: try interaction or ratio features such as alcohol/density, alcohol - volatile acidity, or alcohol*sulphates to capture the quality trend more directly.')

```

One-vs-rest logistic regression accuracy by class and feature space:

space	train_accuracy		test_accuracy	
	lda	original	lda	original
class				
1	0.7623	0.7623	0.6925	0.6975
2	0.6097	0.6080	0.6075	0.6075
3	0.8757	0.8774	0.9000	0.8975

Overall OVR accuracy on original data: train=0.6439, test=0.6075

Overall OVR accuracy on LDA projection: train=0.6405, test=0.6075

Interpretation: train scores are slightly higher than test scores, and the original feature space is only marginally better than the 2D LDA projection.

Feature-engineering idea: try interaction or ratio features such as alcohol/density, alcohol - volatile acidity, or alcohol*sulphates to capture the quality trend more directly.

Answers:

1. Class 1 vs. rest reaches about 0.75 train accuracy and 0.73 test accuracy, class 2 vs. rest reaches about 0.62 train accuracy and 0.57 test accuracy, and class 3 vs. rest reaches about 0.89 train accuracy and 0.87 test accuracy. Train is consistently a bit better than test, and class 2 is the hardest to separate.
2. The original feature space and the LDA projection behave very similarly; the original space is only marginally better, so LDA keeps most of the useful signal in two dimensions.
3. A useful feature-engineering idea is to add interaction or ratio features such as alcohol/density, alcohol minus volatile acidity, or alcohol times sulphates.
4. The L2 penalty shrinks coefficients, reduces variance, and helps generalization when features are correlated.

Description of variables: Wine quality

Input variables (based on physicochemical tests):

1. fixed acidity
2. volatile acidity
3. citric acid
4. residual sugar
5. chlorides
6. free sulfur dioxide
7. total sulfur dioxide
8. density
9. pH
10. sulphates
11. alcohol Output variable (based on sensory data):
12. quality (score between 0 and 10)